

COM 124 – DATA STRUCTURES AND ALGORITHMS

General Objectives: On completion of this course the student should be able to:

- 1.0 Understand concepts of data structure and tools for studying data: symbols, relations and graph.
- 2.0 Know sets relations and string structure.
- 3.0 Know data life cycle data representation, properties of ordered and occupancy.
- 4.0 Know the properties of order and linear list.
- 5.0 Know simple linked lists.
- 6.0 Know non-linear structures.
- 7.0 Understand different sorting and searching techniques

CONCEPTS OF DATA STRUCTURE AND TOOLS

Data: Data are simply collection of facts and figures. Data are values or set of values. A data item refers to a single unit of values. It is a raw fact which becomes information after processing.

Structure: *The arrangement of relations between the parts or elements of something complex. A structure is something if many parts that is put together as structure of body, sentence etc.*

Algorithm: A finite sequence of instructions, each of which has a clear meaning and can be executed with a finite amount of effort in finite time, whatever the input values, an algorithm will definitely terminate after executing a finite number of instructions.

Data Structure

The logical or mathematical model of a particular organization of data is called its data structures. A data item is a single unit of values. It is a raw fact which becomes information after processing. Data items for example, date are called group items if they can be divided into subsystems.

Need for Data Structure

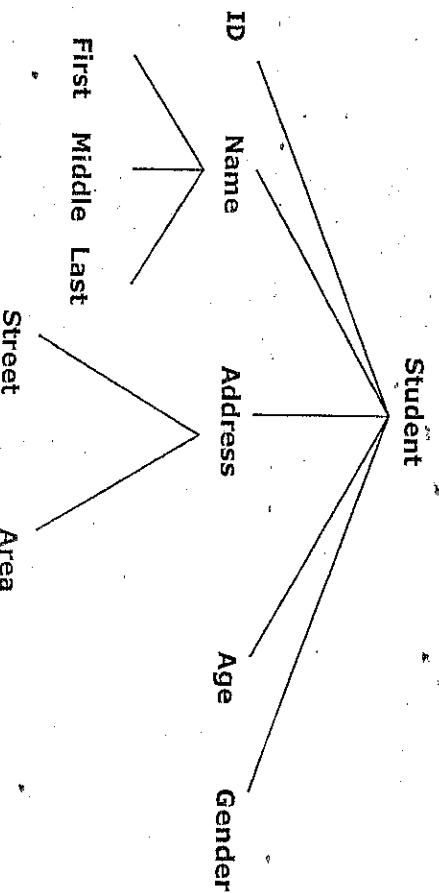
As applications are getting complex and data rich, there are three common problems that applications face now-a-days.

- **Data Search** – Consider an inventory of 1 million (10^6) items of a store. If the application is to search an item, it has to search an item in 1 million (10^6) items every time slowing down the search. As data grows, search will become slower.
 - **Processor Speed** – Processor speed although being very high, falls limited if the data grows to billion records.
 - **Multiple Requests** – as thousands of users can search data simultaneously on a web server, even the fast server fails while searching the data.
- To solve the above-mentioned problems, data structures come to rescue. Data can be organized in a data structure in such a way that all items may not be required to be searched, and the required data can be searched almost instantly.

Data:

Data are simply collection of facts and figures. Data are values or set of values.
A data item refers to a single unit of values.

Data items that are divided into sub items are group items; those that are not are called elementary items. For example, a student's name may be divided into three sub items – [first name, middle name and last name] but the ID of a student would normally be treated as a single item.



In the above example (ID, Age, Gender, First, Middle, Last, Street, Area) are elementary data items, whereas (Name, Address) are group data items.

An entity is something that has certain attributes or properties which may be assigned values. The values themselves may be either numeric or non-numeric. Example:

Attributes:	Name	Age	Gender	Social Society number
Values:				
	Hamza	20	M	134-24-5533
	Peter	23	M	234-9988775
	Fatima	20	F	345-7766443

- Entities with similar attributes (e.g. all the employees in an organization) form an entity set.
- Each attribute of an entity set has a range of values, the set of all possible values that could be assigned to the particular attribute.
- The term "*information*" is sometimes used for data with given attributes, of, in other words meaningful or processed data.
- A field is a single elementary unit of information representing an attribute of an entity,
- a record is the collection of field values of a given entity and
- a file is the collection of records of the entities in a given entity set.

Data Structure:

In computer science, a data structure is a particular way of storing and organizing data in a computer's memory so that it can be used efficiently. Data may be organized in many different ways; the logical or mathematical model of a particular organization of data is called a **data structure**. The choice of a particular data model depends on the two considerations first, it must be rich enough in structure to mirror the actual relationships of the data in the real world. On the other hand, the structure should be simple enough that one can effectively process the data whenever necessary.

Categories of Data Structure:

The data structure can be classified in to major types:

- Linear Data Structure
- Non-linear Data Structure

Linear Data Structure:

A data structure is said to be linear if its elements form any sequence. There are basically two ways of representing such linear structure in memory.

- *One way is to have the linear relationships between the elements represented by means of sequential memory location.* These linear structures are called **arrays**.
- *The other way is to have the linear relationship between the elements represented by means of pointers or links.*

These linear structures are called **linked lists**. The common examples of linear data structure are

- **Arrays**
- **Queues**
- **Stacks**
- **Linked lists**

Arrays: The simplest type of data structure is a linear (or one dimensional) array. A list of a finite number n of similar data referenced respectively by a set of n consecutive numbers, usually 1, 2, 3 n , if we choose the name A for the array, then the elements of A are denoted by subscript notation $A_1, A_2, A_3 \dots An$ or by the parenthesis notation $A(1), A(2), A(3) \dots A(n)$ or by the bracket notation $A[1], A[2], A[3] \dots A[n]$

Example: A linear array $A[8]$ consisting of numbers is pictured in following figure.

1	2	3	4	5	6	7	8
---	---	---	---	---	---	---	---

$A[0]$ $A[1]$ $A[2]$ $A[3]$ $A[4]$ $A[5]$ $A[6]$ $A[7]$
 $\text{int } A[8] = \{1, 2, 3, 4, 5, 6, 7, 8\};$

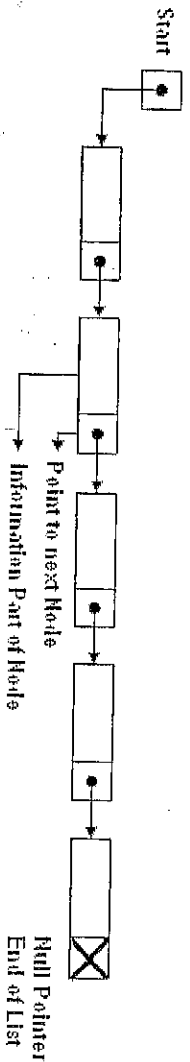
Queue: A queue, also called *FIFO* system, is a linear list in which deletions can take place only at one end of the list, the *Front* of the list and insertion can take place only at the other end *Rear*.

Stack: It is an ordered group of homogeneous items of elements. Elements are added to and removed from the *top* of the stack (the most recently added items are at the top of the stack). The last element to be added is the first to be removed (*LIFO*: Last In, First Out).

Linked List: A linked list, or one way list is a linear collection of data elements, called **nodes**, where the linear order is given by means of pointers. Each **node** is divided into two parts:

- The first part contains the *information* of the element/node
- The second part contains the address of the next node (*link /next pointer field*) in the list.

Example



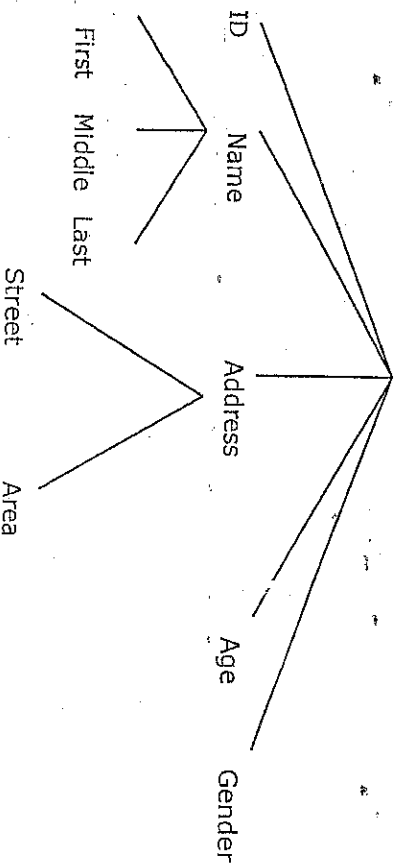
✓ Non-linear Data Structure:

This structure is mainly used to represent data containing a hierarchical relationship between elements. e.g. graphs, family trees and table of contents.

Tree:

Data frequently contain a hierarchical relationship between various elements. The data structure which reflects this relationship is called a **rooted tree graph** or, simply, a **tree**.

Student



Graph:

Data sometimes contains a relationship between pairs of elements which is not necessarily hierarchical in nature, e.g. an airline flights only between the cities connected by lines. This data structure is called **Graph**.

Data Structures Operations:

The data appearing in our data structures are processed by means of certain operations. In fact, the particular data structure that one chooses for a given situation depends largely in the frequency with which specific operations are performed. The following four operations play a major role in this text:

- **Traversing:** accessing each record/node exactly once so that certain items in the record may be processed. (This accessing and processing is sometimes called "visiting" the record.)
- **Searching:** Finding the location of the desired node with a given key value, or finding the locations of all such nodes which satisfy one or more conditions.
- **Inserting:** Adding a new node/record to the structure.
- **Deleting:** Removing a node/record from the structure.

Algorithm. A finite sequence of instructions, each of which has a clear meaning and can be executed with a finite amount of effort in finite time. Whatever the input values, an algorithm will definitely terminate after executing a finite number of instructions.

Characteristics of algorithm

- Has a finite set of steps with definite instructions.
- Instructions have definite order.
- Algorithm must eventually stop.
- Actions are deterministic.

Data Type.

Data type of a variable is the set of values that the variable may assume. While doing programming in any programming language, you need to use various variables to store various information. Variables are nothing but reserved memory locations to store values. This means that when you create a variable you reserve some space in memory.

You may like to store information of various data types like character, wide character, integer, floating point, double floating point, Boolean etc. Based on the data type of a variable, the operating system allocates memory and decides what can be stored in the reserved memory.

- Basic data types in C: **int char float double**
- Basic data types in Pascal: **integer real char boolean**

Type checking:

The main objective of type checking is to determine before program execution whether a domain incompatibility can occur. If so, error messages occur for coercion or execution time testing may be generated.

Type checking can be performed statically (before execution or during compilation) or dynamically (at execution time)

For example, static type checking occurs in most languages such as FORTRAN, COBOL, PASCAL. Dynamic type checking occurs in APL.

Static type checking has following advantages over dynamic type checking.

Advantages of Type checking:

- Efficiency: since the program is typically executed many times but needs only be type-checked once.
- Furthermore type information may use by implementation to improve efficiency in many other ways.
- Minor programming errors could be detected before actual execution; this will simplify program testing and debugging e.g. when another operation is to be performed on non-numerical data.
- Type specification also improves program readability by making explicit the data representation used by the programmer.

Disadvantages:

Syntax of a type language are usually more complex Inflexibility: restriction are imposed on the programmer's freedom of expression.

In conclusion, static type checking should be used in languages to prevent domain incompatibility whenever the disadvantages outweigh its benefit.

Primitive Data Types

A new, primitive type is definable by enumerating the distinct values belonging to it. Such a type is called an *enumeration type*. Its definition has the form

TYPE T = (c1, c2, ..., cn)

T is the new type identifier, and the ci are the new constant identifiers.

Examples

TYPE shape = (rectangle, square, ellipse, circle)

TYPE color = (red, yellow, green)

TYPE sex = (male, female)

TYPE weekday = (Monday, Tuesday, Wednesday, Thursday, Friday, Saturday, Sunday)

TYPE currency = (franc, mark, pound, dollar, shilling, lira, guilder, krone, ruble, cruzeiro, yen, naira)

TYPE vehicle = (train, bus, automobile, boat, airplane)

TYPE rank = (private, corporal, sergeant, lieutenant, captain, major, colonel, general)

TYPE object = (constant, type, variable, procedure, module)

TYPE structure = (array, record, set, sequence)

TYPE condition = (manual, unloaded, parity, skew)

The definition of such types introduces not only a new type identifier, but at the same time the set of identifiers denoting the values of the new type. These identifiers may then be used as constants throughout the program, and they enhance its understandability considerably.

The following are the units for identifying data character, fields, subfields, records, files.

- A file is a collection of logically related records; e.g students file, stock file.
- A record is a collection of logically related data fields; e.g Data relating to students in students file.
- A field is consecutive storage position of values. It is a unit of data within a record e.g student's number, Name, Age. In a database concept fields are usually in columns of a given table.
- Data items for example, date are called group items if they can be divided into subsystems. The date for instance is represented by the day, the month and year; a number is called an elementary item, because it cannot be sub-divided into sub-items otherwise known as Sub fields. It is indeed treated as a single item.
- Character is the smallest unit of information. It includes letters, digits and special symbols such as + (Plus sign), _ (minus sign), \ /, \$, a, b, ..., z, A, B, ..., Z etc. Every character requires one byte of memory unit for storage in computer system.

GRAPH

Introduction

Graphs are a commonly used data structure because they can be used to model many real-world problems. A graph consists of a set of nodes with an arbitrary number of connections, or edges, between the nodes. These edges can be either directed or undirected and weighted or unweighted.

In this study we will examine the basics of graphs and create a Graph class. This class was similar to the Binary Tree class, the difference being that instead of only have a reference for at most two edges, the Graph class's Graph Nodes could have an arbitrary number of references. This similarity is not surprising because trees are a special case of graphs definitions

Definition 1. A *graph* $G = (V, E)$ is a finite nonempty set V of objects called *vertices* (the singular is *vertex*) together with a (possibly empty) set E of unordered pairs of distinct vertices of called *edges*.

Graphs are a very expressive formalism for system modeling, especially when attributes are allowed. Our research is mainly focused on the use of graphs for system verification.

Definition 2. A *digraph* $G = (V, E)$ is a finite nonempty set V of vertices together with a (possibly empty) set E of ordered pairs of vertices of G called *arcs*.

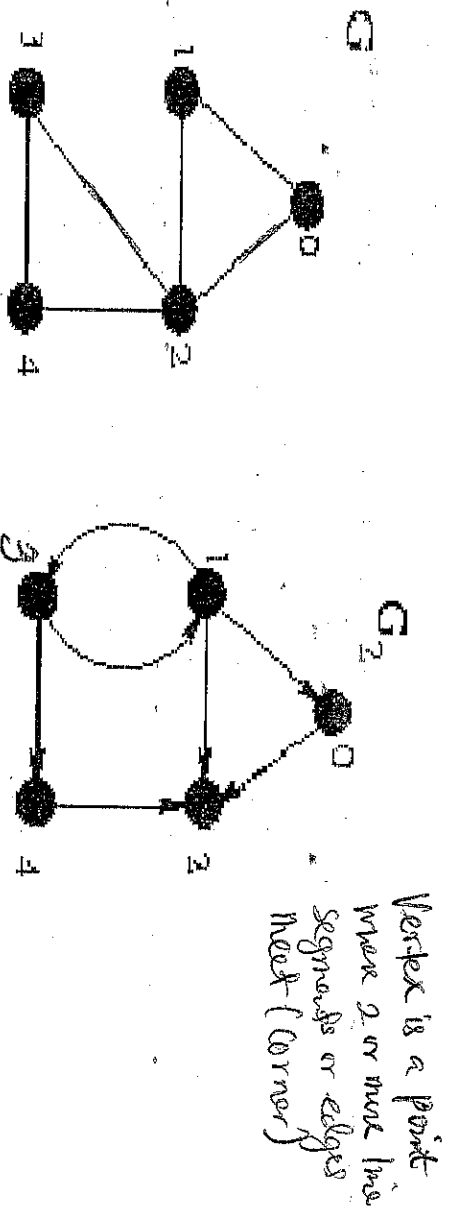
An arc that begins and ends at a same vertex u is called a *loop*. We usually (but not always) disallow loops in our digraphs. By being defined as a set, E does not contain duplicate (or multiple) edges/arcs between the same two vertices. For a given graph (or digraph) G we also denote the set of vertices by $V(G)$ and the set of edges (or arcs) by $E(G)$ to lessen any ambiguity.

Definition 3. The order of a graph (digraph) $G = (V, E)$ is $|V|$, sometimes denoted by $|G|$, and the size of this graph is $|E|$.

Sometimes we view a graph as a digraph where every unordered edge (u, v) is replaced by two directed arcs (u, v) and (v, u) . In this case, the size of a graph is half the size of the corresponding digraph.

In the next example we display a graph G_1 and a digraph G_2 both of order 5. The size of the graph G_1 is 6 where $E(G_1) = \{(0, 1), (0, 2), (1, 2), (2, 3), (2, 4), (3, 4)\}$ while the size of the digraph G_2 is 7 where $E(G_2) = \{(0, 2), (1, 0), (1, 2), (1, 3), (3, 1), (3, 4), (4, 2)\}$.

Example 4. A pictorial example of a graph G_1 and a digraph G_2 is given below.



Definition 5. A walk in a graph (digraph) G is a sequence of vertices $v_0, v_1, \dots, v_n$ such that, for all

$0 \leq i < n$, (v_i, v_{i+1}) is an edge (arc) in G . The *length* of the walk $v_0, v_1, \dots, v_n$ is the number n (i.e., number of edges/arcs). A *path* is a walk in which no vertex is repeated. A *cycle* is a walk (of length at least three for graphs) in which $v_0 = v_n$ and no other vertex is repeated; sometimes, if it is understood, we omit v_n from the sequence.

Example 6. For the graph G_1 of Example 4 the following sequences of vertices are classified as being walks, paths, or cycles.

$v_0, v_1, \dots$	is walk?	is path?	is cycle?
0 1 2 3 4	yes	yes	no
0 1 2 0	yes	no	yes
0 1 2	yes	yes	yes (understand)
0 3 2	no	no	no
0 1 0	yes	no	no

walk:
cycle:
paths: a way or track laid down
for walking or made by walking
and the ending

Example 7. For the digraph G_2 of Example 4 the following sequences of vertices are classified as being walks, paths, or cycles.

$v_0, v_1, \dots$	is walk?	is path?	is cycle?
0 1 2 3 4	no	no	no
0 2 4	no	no	no
3 1 2	yes	yes	no
1 3 1	yes	no	yes
3 1 3 1 0	yes	no	no

Definition 8. A graph G is *connected* if there is a path between all pairs of vertices u and v of $V(G)$. A digraph G is *strongly connected* if there is a path from vertex u to vertex v for all pairs u and v in $V(G)$.

In Example 4 the graph G_1 is connected but the digraph G_2 is not strongly connected because there are no arcs leaving vertex 2. The *underlying graph* (by replacing each arc with an edge) of G_1 is connected, however.

Definition 9. In a graph, the *degree* of a vertex, denoted by $\deg(v)$, is the number of edges incident to v . For digraphs, the *out-degree* of a vertex v is the number of arcs incident from v (leaving v) and the *in-degree* of vertex v is the number of arcs incident to v (entering v).

For a graph the in-degree and out-degrees are the same as the degree. For our graph, we have $\deg(0)=2$, $\deg(1)=2$, $\deg(2)=4$, $\deg(3)=2$ and $\deg(4)=2$. We may concisely write this as a *degree sequence* (2, 2, 4, 2, 2) if there is a natural ordering (e.g., 0, 1, 2, 3, 4) of the vertices. The in-degree sequence and out-degree sequence of the digraph are (1, 1, 3, 1, 1) and (1, 3, 0, 2, 1), respectively. The degree of a vertex of a digraph is sometimes defined as the sum of its in-degree and out-degree. Using this definition, a degree sequence of G_2 would be (2, 4, 3, 3, 2).

Definition 10. The *diameter* of a connected graph (strongly connected digraph) $G = (V, E)$ is the least integer D such that for all vertices u and v in G we have $\leq D$, where $d(u, v)$ denotes the *distance* from u to v in G , that is, the length of a shortest path between u and v .

Example 11. The diameter of graph G_1 of Example 4 is 2. We calculated $d(0,1)=1$, $d(0,2)=1$, $d(0,3)=2$, $d(0,4)=2$, $d(1,2)=1$, $d(1,3)=2$, $d(1,4)=2$, $d(2,3)=1$, $d(2,4)=1$ and $d(3,4)=1$. Note for graphs, we have $d(x, y) = d(y, x)$ for all vertices x and y .

Since the digraph G_2 is not strongly connected the diameter is undefined. However, we can compute shortest distances between various pairs of vertices: $d(0,2)=1$, $d(1,0)=1$, $d(1,2)=1$, $d(3,1)=1$, $d(3,0)=2$, $d(3,2)=2$, $d(3,4)=1$ and $d(4,2)=1$.

Computer representations of graphs

There are two common computer representations for graphs (or digraphs), called adjacency matrices and adjacency lists. For a graph G of order n , an adjacency matrix representation is a Boolean matrix (often encoded with 0's and 1's) of dimension n such that entry (i,j) is true if and only if edge/arc (i,j) is in $E(G)$. For a graph G of order n , an adjacency lists representation is n lists such that the i^{th} list contains a sequence (often sorted) of out-neighbours of vertex i of G .

We can see the structure of these representations more clearly with examples.

Example 12. We give adjacency matrices for the graph G_1 and digraph G_2 of Example 4 below.

$$\begin{bmatrix} 0 & 1 & 1 & 0 & 0 \\ 1 & 0 & 1 & 0 & 0 \\ 1 & 1 & 0 & 1 & 1 \\ 0 & 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 1 & 0 \end{bmatrix} \quad \begin{bmatrix} 0 & 0 & 1 & 0 & 0 \\ 1 & 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 & 0 \end{bmatrix}$$

Notice that the 1's in the rows represent how many out-neighbours and the 1's in the columns represent how many in-neighbours a vertex has.

Example 13. We give adjacency lists for the graph G_1 and digraph G_2 of Example 4 below.

0:	1	2	0:	1	2				
1:	0	2	1:	0	2				
2:	0	1	3	4	2:	0	1	3	4
3:	2	4	3:	2	4				
4:	2	3	4:	2	3				

Only the out-neighbours are listed in the adjacency lists representation. The numbers with colons (:) denote the index i of the lists (and are not really necessary). An empty list can occur (e.g., list 2 of the digraph G_2).

For a graph/digraph with n vertices and m edges, the adjacency matrix representation requires $O(n^2)$ storage while the adjacency lists representation requires $O(m)$ storage. So for sparse graphs the latter is probably preferable. However, to check whether edge/arc (i,j) is in the graph the adjacency matrix representation has constant-time lookup, while the adjacency lists representation may require $O(n)$ time in the worst case.

We mention that there are also other specialized graph representations besides the two mentioned in this section. These data structures take advantage of the graph structure for improved storage or access time, often for families of graphs sharing a common property.

SYMBOL AND RELATIONS

A symbol is something such as an object, picture, written word, sound, or particular mark that represents something else by association, resemblance, or convention. For example, a red octagon may stand for "STOP". On maps, crossed sabres may indicate a battlefield. Numerals are symbols for numbers.

Language and symbols

All languages are made up of symbols. Spoken words are the symbols of mental experience, and written words are the symbols of spoken words. The word "cat", for example, whether spoken or written, is not a literal cat but a sequence of symbols that by convention associate the word with a concept. Hence, the written or spoken word "cat" represents (or stands for) a particular concept formed in the mind. A drawing of a cat, or a stuffed cat, could also serve as a symbol for the idea of a cat. The study or interpretation of symbols is known as *symbolology*, and the study of signs is known as *semiotics*.

Relational Operation Symbols

[illegible]

Explore equation tools in MS-Word for the names of the above symbols

String structure.

- Define string.
- Explain basic operations of strings.

String Processing

A finite sequence S of zero or more characters is called a string. The number of character in a string is called its length. The string with zero character is called the empty string or the null string. The following are strings of length 9, 18, 14 and 0 respectively:

- I.) "NDI CLASS"
II.) "COMPUTER DEPARTMENT"

- III.) "CAMPUS SHUTTLE", and
- IV.) ""

Concatenation of Strings

Let S1 and S2 be strings, the string consisting of the characters of S1 followed by the characters of S2 is called concatenation of S1 and S2.

This is denoted by S1//S2. E.g. "STARLETS"//"/"DEFEAT"/"EA.GLET"

A string Y is called a substring of a string S if there exist strings X and Z such that S=X/Y/Z.

If X is an empty string, then Y is called an initial substring of S, and if Z is an empty string then Y is called a terminal substring of S.

E.g. 'BE OR NOT' is a substring of 'TO BE OR NOT TO BE'.

'THE' is an initial substring of 'THE END'.

Length

The general form is LENGTH(string) and this will return the number of character(s) in a giving string.

- i.) LENGTH('student')=7
- ii.) LENGTH('')=0

Insertion

Suppose we want to insert a string in a given text T so that S starts in position K. We denote this operation as INSERT(text, position, string)

E.g. INSERT('ABCDEFG', 3, 'XYZ')='ABXYZCDEFG'

Deletion

The general form is DELETE(text, position, length).

E.g. DELETE('ABCDEFG', 4, 2)='ABCFG'

We assume that nothing is deleted if position K=0.

Thus DELETE('ABCDEFG', 0, 2)='ABCDEFG'

Suppose that text T and pattern P are given and it is required to delete from T the first occurrence of the pattern P. We can use the following DELETE function

DELETE(T, INDEX(T, P), LENGTH(P))

E.g. = 'ABCDEFG', P='CD', then

DELETE(T, INDEX(T, P), LENGTH(P))=DELETE('ABCDEFG', INDEX('ABCDEFG', 'CD'), 2)='ABCFG'

Suppose that we want to delete every occurrence of the pattern P in the text T, then we can do this by repeatedly applying DELETE(T, INDEX(T, P), LENGTH(P))

Until INDEX(T, P)=0 that is, until P does not appear in T.

The following algorithm is used to accomplish this:

Algorithm:

A text T and a pattern P are in computer memory. This algorithm deletes every occurrence of P in T.

- i.) Find index of P in T. Set K=INDEX(T, P)
- ii.) Repeat while K not equal to 0
 - a. [Delete P from T.]
- Set T:= DELETE(T, INDEX(T, P), LENGTH(P))
- b. [Update index]
- Set K:= INDEX(T, P)
- [End of loop]
- iii.) Write: T.
- iv.) Exit.

Replacement

Suppose in a given T we want to replace the first occurrence of a pattern P1 by a pattern P2, we will denote this operation by REPLACE(text, pattern1, pattern2)

E.g. REPLACE('XABYABZ','AB','C')='XCZYABZ'
REPLACE('XABYABZ','BA','C')='XABYABZ'

In the second case, the pattern BA does not occur, and hence there is no change. Suppose a text T and patterns P and Q are in the memory of a computer. Suppose we want to replace every occurrence of the pattern P in T by the Pattern Q. This might be accomplished by repeatedly applying REPLACE(T,P,Q), UNTIL(T,P)=0
This could be done using the following algorithm:

1. [Find index of P] Set K := INDEX(T,P)
2. Repeat while K>0:
 - a.) [Replace P by Q] Set T:=REPLACE(T,P,Q)
 - b.) [Update] Set K:= INDEX(T,P)
- [End loop]
3. Write: T
4. Exit.

Exercise

- a.) T=XABYABZ
P=AB
Q=C
REPLACE(T,P,Q)
- b.) IfT=XAB
P=A
Q=AB
REPLACE(T,P,Q)

Use the algorithm above to solve the problems.

ARRAYS IN C (Single Dimension)

Arrays are preferred for situations which require similar type of data items to be stored together.

An array is a **finite** collection of similar elements stored in adjacent memory locations. By **finite** we mean that there are specific number of elements in an array and **similar** implies that all the elements in an array are of the same type. For example, an array may contain all integers or all characters.

Thus an array is a collection of variables of the same type that are referred by a common name. The elements of the array are referenced respectively by an **index set** containing n consecutive numbers. An array with a number of elements is referenced using an index that ranges from 0 to $n-1$. The lowest index of an array is called its **lower bound** and highest index is called the **upper bound**. The number of elements in an array is called its **range**. The elements of an array $A[n]$ containing n elements are referenced as $A[0]$, $A[1]$, $A[2]$, ..., $A[n-1]$ where 0 is the lower bound and ' $n-1$ ' is the upper bound of the array.

For example :

```
int A[10] = {5, 6, 12, 4, 15, 45, 07, 1, 8, 19};
```

A[0]	A[1]	A[2]	A[3]	A[4]	A[5]	A[6]	A[7]	A[8]	A[9]	Subscript Index
5	6	12	4	15	45	87	1	9	13	
100	104	108	112	116	120	124	128	132	136	Addresses

Representation of Linear Arrays in Memory

The memory of computer is simply a sequence of addressed locations as shown in Fig. Let A be a linear array stored in the memory of computer as given in above figure :

Addr($A[k]$) = address of the $A[k]$ element of the array A

As the elements of the array A are stored in consecutive memory cells, the computer does not need to keep track of the address of every element of the array. It only needs to keep track of the address of the first element of the array which is denoted by:

$$\text{Base}(A)$$

It is called the base address of the A . Using this base address $\text{Base}(A)$, the computer calculates the address of any element of the array by using the following formulae:

$$\text{Addr}(A[k]) = \text{Base}(A) + w(k - \text{lower bound})$$

where w is the size of the data type of the array A and k is the index number.

Using the figure given on top of this page we are calculating the address of $A[5]$

$$\begin{aligned} \text{Addr}(A[5]) &= 100 + 4(5 - 0) \\ &= 120 \end{aligned}$$

Declaration of the Arrays: Any array declaration contains: the *array name*, the *element type* and the *array size*.

Examples:

```
int a[20], b[3], c[7];
float f[5], c[2];
char m[4], n[20];
```

Initialization of an array is the process of assigning initial values. Typically declaration and initialization are combined.

Examples:

```
float, b[3]={2.0, 5.5, 3.14};
char name[4]={ 'E', 'm', 'i', 'e' };
int c[10]={0, 1, 2, 3, 4, 5, 6, 7, 8, 9};
```

Operations on array

- 1- **Traversing:** means to visit all the elements of the array in an operation is called traversing.
- 2- **Insertion:** means to put values into an array
- 3- **Deletion / Remove:** to delete a value from an array.
- 4- **Sorting:** Re-arrangement of values in an array in a specific order (Ascending / Descending) is called sorting.
- 5- **Searching:** The process of finding the location of a particular element in an array is called searching. There are two popular searching techniques/mechanisms:

a) Traversing in Linear Array:

It means processing or visiting each element in the array exactly once;
Let 'A' is an array stored in the computer's memory. If we want to display the contents of 'A', it has to be traversed, i.e. by accessing and processing each element of 'A' exactly once.

Algorithm:

(Traverse a Linear Array) Here LA is a linear array with lower boundary LB and upper boundary UB. This algorithm traverses LA applying an operation Process to each element of LA.

1. [Initialize counter.] Set K=LB.
2. Repeat Steps 3 and 4 while K≤UB.
3. [Visit element.] Apply PROCESS to LA[K].
4. [Increase counter.] Set K=K+1.
- [End of Step 2 loop.]
5. Exit.

This program will traverse each element of the array to calculate the sum and then calculate & print the average of the following array of integers. (4, 3, 7, -1, 7, 2, 0, 4, 2, 13)

```
#include <iostream>
using namespace std;
#define size 10 // another way int const size = 10
int main()
{ int x[size]={4,3,7,10,7,2,0,4,2,13}, i, sum=0, LB=0, UB=size;
  float av;
  for(i=LB; i<UB; i++) sum = sum + x[i];
  av = sum/size;
  cout << "The sum of the numbers=" << sum << endl;
  cout << "The average of the numbers=" << av << endl;
  return 0;
}
```

b) Sorting in Linear Array:

Sorting an array is the ordering the array elements in *ascending* (increasing - from min to max) or *descending* (decreasing - from max to min) order.

Example:

{2 1 5 7 4 3} □ {1, 2, 3, 4, 5, 7} *ascending* order
 {2 1 5 7 4 3} □ {7, 5, 4, 3, 2, 1} *descending* order

Bubble Sort:

The technique we use is called "*Bubble Sort*" because the bigger value gradually bubbles their way up to the top of array like air bubble rising in water, while the small values sink to the bottom of array. This technique is to make several passes through the array. On each pass, successive pairs of elements are compared. If a pair is in increasing order (or the values are identical), we leave the values as they are. If a pair is in decreasing order, their values are swapped in the array.

Bubble Sort

Pass=1	Pass=2	Pass=3	Pass=4
<u>2</u> 1 5 7 4 3	<u>1</u> 2 5 4 3 7	<u>1</u> 2 4 3 5 7	<u>1</u> 2 3 4 5 7
1 <u>2</u> 5 7 4 3	1 <u>2</u> 5 4 3 7	1 <u>2</u> 4 3 5 7	1 <u>2</u> 3 4 5 7
1 2 <u>5</u> 7 4 3	1 2 <u>5</u> 4 3 7	1 2 4 <u>3</u> 5 7	1 2 3 4 <u>5</u> 7
1 2 5 <u>7</u> 4 3	1 2 5 4 <u>3</u> 7	1 2 4 3 <u>5</u> 7	1 2 3 4 5 <u>7</u>
1 2 5 4 <u>7</u> 3	1 2 4 <u>5</u> 3 7	1 2 3 4 <u>5</u> 7	
1 2 5 4 7 <u>3</u>	1 2 4 3 <u>5</u> 7		
1 2 5 4 3 7			

Algorithm: (Bubble Sort) BUBBLE (DATA, N)

Here DATA is an Array with N elements. This algorithm sorts the elements in DATA.

```

1. for pass=1 to N-1.
2.   for (i=0; i<= N-Pass; i++)
3.     IF DATA[i]>DATA[i+1], then:
       Interchange DATA[i] and DATA[i+1].
       [End of If Structure.]
       [End of inner loop.]
       [End of Step 1 outer loop.]
4. Exit.
/* this program sorts the array elements in the ascending order using bubble sort method */
#include <iostream>
using namespace std;
int const SIZE = 6;
int main()
{
    int a[SIZE]= {77,42,35,12,101,6};
    int i;
    cout << "The elements of the array before sorting\n";
    for (i=0; i<= SIZE-1; i++)

```

```

    {cout<<a[i]<<" ";
    }

    int pass, hold;
    for (pass=1; pass<= SIZE-1; pass++)
    {
        for (i=0; i<= SIZE-pass; i++)
        {
            if(a[i] >a[i+1])
            {
                hold =a[i];
                a[i]=a[i+1];
                a[i+1]=hold;
            }
        }
    }

    cout<< "\n\nThe elements of the array after sorting\n";
    for (i=0; i<= SIZE-1; i++)
    {
        cout<< a[i]<< " ";
    }
}

```

return 0;

}

Home Work

Write a program to determine the median of the array given below:
(9, 5, 1, 7, 78, 22, 15, 96, 45, 25)

Note that the median of an array is the middle element of a sorted array.

c) Searching in Linear Array:

The process of finding a particular element of an array is called *Searching*". If the item is not present in the array, then the search is unsuccessful. There are two types of search (*Linear search* and *Binary Search*)

Linear Search:

The linear search compares each element of the array with the *search key* until the search key is found. To determine that a value is not in the array, the program must compare the search key to every element in the array. It is also called "*Sequential Search*" because it traverses the data sequentially to locate the element.

Algorithm: (Linear Search)"

LINEAR (A, SKEY)

Here A is a Linear Array with N elements and SKEY is a given item of information to search. This algorithm finds the location of SKEY in A and if successful, it returns its location otherwise it returns -1 for unsuccessful.

1. Repeat for i = 0 to N-1
2. if (A[i] = SKEY) return i [Successful Search]
3. return -1 [Un-Successful]
4. Exit.

/ This program use linear search in an array to find the LOCATION of the given Key value */*

/ This program is an example of the Linear Search */*


```

#include <iostream>
using namespace std;
int const N=10;
int LinearSearch(int [], int); // Function Prototyping
int main()
{
    int A[N]={9, 4, 5, 1, 7, 78, 22, 15, 96, 45}, Skey, LOC;
    cout<< " Enter the Search Key\n";
    cin>>Skey;
    LOC = LinearSearch( A, Skey); // call a function
    if(LOC == -1)
        cout<< " The search key is not in the array\n Un-Successful Search\n";
    else
        cout<< " The search key "<<Skey<<" is at location " <<LOC<<endl;
    return 0;
}
int LinearSearch( int bf [], int skey) // function definition
{
    int i;
    for (i=0; i<= N-1; i++) if(bf[i] == skey) return i;
    return -1;
}

```

Binary Search:

It is useful for the large sorted arrays. The binary search algorithm can only be used with *sorted array* and eliminates one half of the elements in the array being searched after each comparison. The algorithm locates the middle element of the array and compares it to the search key. If they are equal, the search key is found and array subscript of that element is returned. Otherwise the problem is reduced to searching one half of the array. If the search key is less than the middle element of array, the first half of the array is searched. If the search key is not the middle element of in the specified sub array, the algorithm is repeated on one quarter of the original array. The search continues until the sub array consist of one element that is equal to the search key (search successful). But if Search-key not found in the array then the value of END of new selected range will be less than the START of new selected range.

Algorithm: (Binary Search)

Here A is a sorted Linear Array with N elements and SKEY is a given item of information to search. This algorithm finds the location of SKEY in A and if successful, it returns its location otherwise it returns -1 for unsuccessful.

Binary Search (A, SKEY)

1. [Initialize segment variables.]

Set START=0, END=N-1 and MID=INT((START+END)/2).

2. Repeat Steps 3 and 4 while START ≤ END and A[MID]≠SKEY.

3. If SKEY < A[MID]. Then

Set END=MID-1.

Else Set START=MID+1.

[End of If Structure.]

4. Set MID=INT((START+END)/2).

[End of Step 2 loop.]

5. If A[MID]=SKEY then Set LOC= MID

Else:

Set LOC = -1

[End of IF structure.]

6. return LOC and Exit

```
// C++ Code for Binary Search
#include <iostream>
using namespace std;
int const N=10;
int BinarySearch(int l, int); // Function Prototyping
int main()
{
    int A[N]={3, 5, 9, 11, 15, 17, 22, 25, 37, 68}, SKEY, LOC;
    cout<< " Enter the Search Key\n ";
    cin>>SKEY;
    LOC = BinarySearch(A, SKEY); // Function call
    if(LOC == -1)
        cout<< " The search key is not in the array\n ";
    else
        cout<< " The search key " <<SKEY << " is at location " <<LOC<<endl;
    return 0;
}

int BinarySearch (int A[], int skey)
{
    int START=0, END=N-1, MID=int((START+END)/2), LOC;
    while(START <= END && A[MID] != skey)
    {
        if(skey < A[MID])
            END = MID - 1;
        else
            START = MID + 1;
        MID=int((START+END)/2);
    }
    if(A[MID] == skey)
        LOC=MID;
    else LOC=-1;
    return LOC;
}
```

Computational Complexity of Binary Search

The *Computational Complexity* of the *Binary Search* algorithm is measured by the maximum (worst case) number of Comparisons it performs for searching operations. The searched array is divided by 2 for each comparison/iteration.

Therefore, the maximum number of comparisons is measured by: $\log_2(n)$ where n is the size of the array

Example:

If a given sorted array 1024 elements, then the maximum number of comparisons required is: $\log_2(1024) = 10$ (only 10 comparisons are enough)

Computational Complexity of Linear Search

Note that the *Computational Complexity* of the *Linear Search* is the maximum number of comparisons you need to search the array. As you are visiting all the array elements in the worst case, then, the number of comparisons required is: n (n is the size of the array)

Example:

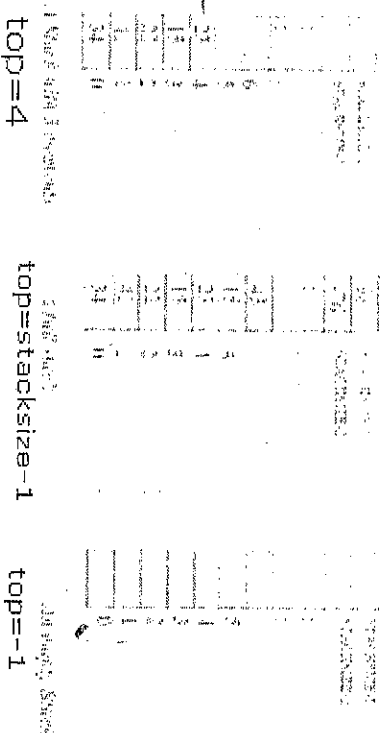
If a given array of 1024 elements, then the maximum number of comparisons required is: $n-1 = 1023$ (As many as 1023 comparisons may be required)

STACKS:

It is an ordered group of homogeneous items of elements. Elements are added to and removed from the top of the stack (the most recently added items are at the top of the stack). The last element to be added is the first to be removed (LIFO: Last In, First Out).

A stack is a list of elements in which an element may be inserted or deleted only at one end, called *TOP* of the stack. The elements are removed in reverse order of that in which they were inserted into the stack.

200



Basic operations:

These are two basic operations associated with stack:

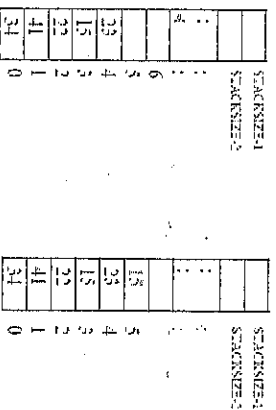
Push 0 is the term used to insert/add an element into a stack.

Pop 0 is the term used to delete/remove an element from a stack.

Other names for stacks are *piles* and *push-down lists*.

Push Operation

Push an item onto the top of the stack (*insert an item*)



Pop Operation

Pop an item off the top of the stack (delete an item)

Write: OVERFLOW, and Return.

2. [Find new value of REAR.]

 If COUNT=0, then: [Queue initially empty.]

 Set FRONT=0 and REAR=0

 Else: if REAR = MAXSIZE - 1, then:

 Set REAR = 0

 Else:

 Set REAR = REAR+1.

 [End of If Structure.]

3. Set QUEUE[REAR] = ITEM. [This insert new element.]

4. COUNT=COUNT+1 [Increment to Counter.]

5. Return.

Deletion in Queue:

Algorithm: DEQUEUE(QUEUE, MAXSIZE, FRONT, REAR, COUNT, ITEM)

 This procedure deletes an element from a queue and assigns it to the variable ITEM.

1. [QUEUE already empty?]

 If COUNT=0, then: Write: UNDERFLOW, and Return.

2. Set ITEM = QUEUE[FRONT].

3. Set COUNT = COUNT - 1

4. [Find new value of FRONT.]

 If COUNT = 0, then: [There was one element and has been deleted.]

 Set FRONT=-1, and REAR = -1.

 Else if FRONT= MAXSIZE, then: [Circular, so set Front = 0]

 Set FRONT = 0

 Else:

 Set FRONT:=FRONT+1.

 [End of If structure.]

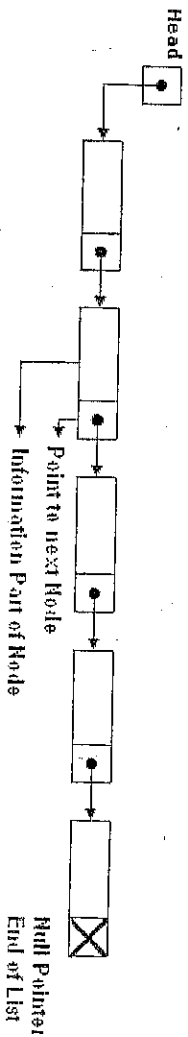
5. Return ITEM

LINKED LIST:

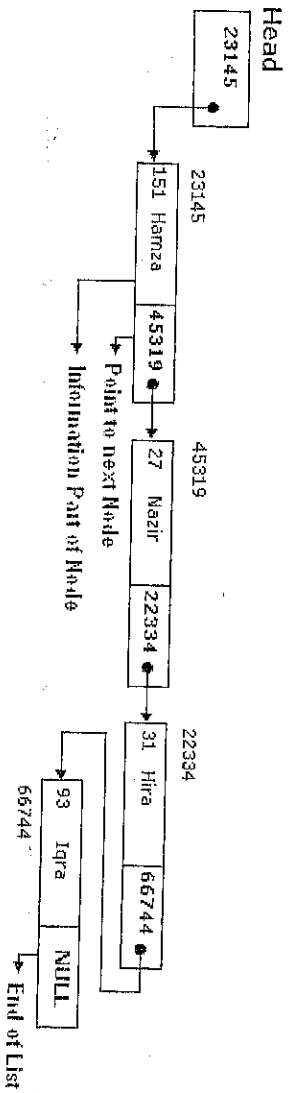
A linked list or one way list is a linear collection of data elements, called *nodes*, where the linear order is given by means of "*pointers*". Each node is divided into two parts.

- The first part contains the information of the element.
- The second part called the link field contains the address of the next node in the list.

To see this more clearly let's look at an example:



For example:



The *Head* is a special pointer variable which contains the address of the first node of the list. If there is no node available in the list then *Head* contains *NULL* value that means, list is empty. The left part, of the each node represents the information part of the node, which may contain an entire record of data (e.g., ID, name, marks, age etc). the right part represents pointer/link to the next node. The next pointer of the last node is *null* pointer signal the end of the list.

Advantages:

List of data can be stored in arrays but linked structures (pointers) provide several advantages.

- A linked list is appropriate when the number of data elements to be represented in data structure is unpredictable.

- It also appropriate when there are frequently insertions & deletions occurred in the list.
- Linked lists are dynamic, so the length of a list can increase or decrease as necessary.

Operations on Linked List:

There are several operations associated with linked list i.e.

a) Traversing a Linked List

Suppose we want to traverse a linked list in order to process each node exactly once.

The traversing algorithm uses a pointer variable PTR which points to the node that is currently being processed. Accordingly, PTR->NEXT points to the next node to be processed so, PTR=HEAD [Moves the pointer to the first node of the list]
PTR=PTR->NEXT [Moves the pointer to the next node in the list.]

Algorithm: (Traversing a linked list) Let LIST be a linked list in memory. This algorithm traverses LIST, applying an operation PROCESS to each element of list. The variable PTR point to the node currently being processed.

1. Set PTR=HEAD. [Initializes pointer PTR.]
2. Repeat Steps 3 and 4 while PTR!=NULL.
3. Apply PROCESS to PTR->INFO.
4. Set PTR= PTR-> NEXT [PTR now points to the next node.]
5. Exit. [End of Step 2 loop.]

b) Searching a Linked List:

Let list be a linked list in the memory and a specific ITEM of information is given to search. If ITEM is actually a key value and we are searching through a LIST for the record containing ITEM, then ITEM can appear only once in the LIST. Search for wanted ITEM in List can be performed by traversing the list using a pointer variable PTR and comparing ITEM with the contents PTR->INFO of each node, one by one of list.

Algorithm: SEARCH(INFO, NEXT, HEAD, ITEM, PREV, CURR, SCAN) LIST is a linked list in the memory. This algorithm finds the location LOC of the node where ITEM first appear in LIST, otherwise sets LOC=NULL.

1. Set PTR=HEAD.
2. Repeat Step 3 and 4 while PTR≠NULL:
3. if ITEM = PTR->INFO then:
 Set LOC=PTR, and return. [Search is successful.]
 [End of If structure.]
4. Set PTR=PTR->NEXT
5. Set LOC=NULL, and return. [Search is unsuccessful.]
 [End of Step 2 loop.]
6. Exit.

Search Linked List for insertion and deletion of Nodes:

Both insertion and deletion operations need searching the linked list.

- □ To add a new node, we must identify the logical predecessor (address of previous node) where the new node is to be inserting.
- □ To delete a node, we must identify the location (addresses) of the node to be deleted and its logical predecessor (previous node).

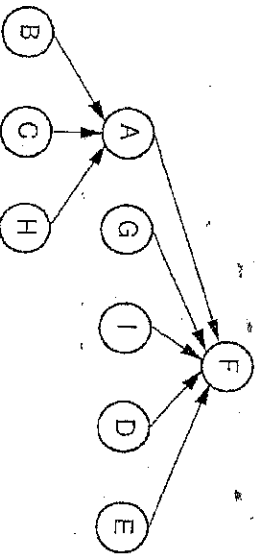
TREE:

So far, we have been studying mainly linear types of data structures: arrays, lists, stacks and queues. Now we defines a nonlinear data structure called *Tree*. This structure is mainly used to represent data containing a hierarchical relationship between nodes/elements e.g. family trees and tables of contents.

- □ General Tree
- □ Binary Tree

General Tree:

A tree where a node can has any number of children / descendants is called General Tree. For example:



Binary Tree:

A tree in which each element may has 0-child, 1-child or maximum of 2-children.

A *Binary Tree* T is defined as finite set of elements, called *nodes*, such that:

- a) T is empty (called the null tree or empty tree.)

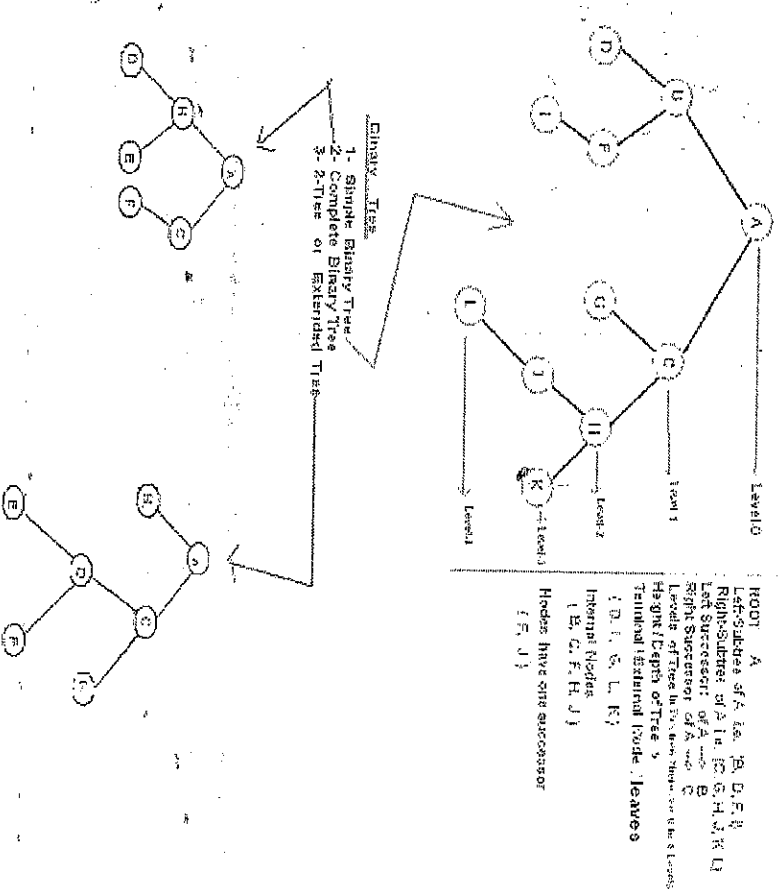
b) T contains a distinguished node R, called the *root* of T, and the remaining nodes of T form an ordered pair of disjoint binary trees T1 and T2.

If T does contain a *root* R, then the two trees T1 and T2 are called, respectively, the *left sub tree* and *right sub tree* of R.

If T1 is nonempty, then its node is called the left successor of R; similarly, if T2 is nonempty, then its node is called the **right successor** of R. The nodes with no successors are called the **terminal nodes**. If N is a node in T with left successor S1 and right successor S2, then N is called the **parent (or father)** of S1 and S2. Analogously, S1 is called the left child (or son) of N, and S2 is called the right child (or son) of N. Furthermore, S1 and S2 are said to **siblings (or brothers)**. Every node in the binary tree T, except the root, has a unique parent, called the predecessor of N. The line drawn from a node N of T to a successor is called an **edge**, and a sequence of consecutive edges is called a **path**. A terminal node is called a **leaf**, and a path ending in a leaf is called a **branch**. The **depth (or height)** of a tree T is the maximum number of nodes in a branch of T. This turns out to be 1 more than the largest level number of T.

Level of node & its generation:

Each node in binary tree T is assigned a level number, as follows. The root R of the tree T is assigned the level number 0, and every other node is assigned a level number which is 1 more than the level number of its parent. Furthermore, those nodes with the same level number are said to belong to the same generation.



Complete Binary Tree:

Consider a binary tree T, each node of T can have at most two children.

Accordingly, one can show that level $n-2$ of T can have at most two nodes.

A tree is said to be **complete** if all its levels, except possibly the last have the maximum number of possible nodes, and if all the nodes at the last level appear as far left as possible.

Extended Binary Tree: 2-Tree:

A binary tree T is said to be a 2-tree or an extended binary tree if each node N has either 0 or 2 children. In such a case, the nodes, with 2 children are called *internal nodes*, and the node with 0 children are called *external node*.

Traversing of Binary Tree:

A traversal of a tree T is a systematic way of accessing or visiting all the node of T. There are three standard ways of traversing a binary tree T with root R. these are:

□□Preorder (N L R):

- Process the node/root.
- Traverse the Left sub tree.
- Traverse the Right sub tree.

(A B D F I C G H J K)

□□Inorder (L N R):

- Traverse the Left sub tree. (D B I F A G C L J H K)
- Process the node/root.
- Traverse the Right sub tree.

□□Postorder (L R N):

- Traverse the Left sub tree. (D I F B G L J K H C A)
- Traverse the Right sub tree.
- Process the node/root.

